Introduction to Git – Installation and Basic Commands

What is Git?

- **History and Development:**
  - Created by Linus Torvalds in 2005, Git was developed to support the Linux kernel development process.
  - Git is a distributed version control system (DVCS) that tracks changes in source code during software development.
  - It is designed to handle small to large projects with speed and efficiency.

Why Git?

- **Advantages over Other VCS:**
  - **Distributed System:** Unlike centralized systems (e.g., SVN), Git allows every developer to have a full history of the repository, enabling offline access and independent work.
  - **Efficient Branching and Merging:** Git's lightweight branches encourage experimentation and smooth merging processes.
  - **Speed:** Operations like commits, branches, and merges are extremely fast due to Git's architecture.
  - **Open Source:** Git is free and open-source, making it accessible to everyone.

Overview of GitHub and Other Git Hosting Services:?

GitHub: A cloud-based platform for hosting Git repositories, offering features like issue tracking, collaboration tools, and integration with CI/CD pipelines.

Other Services:

- **GitLab:** Similar to GitHub but also includes built-in CI/CD tools.
- **Bitbucket:** Focuses on integration with Atlassian tools like Jira and offers free private repositories.

Installing Git

- **Installing Git on Different Operating Systems:**
  - **Windows:**
    - Download Git from the official website (https://git-scm.com/).
    - Follow the installation wizard, selecting appropriate options (e.g., using Git from the command line).
  - **macOS:**
    - Install via Homebrew by running `brew install git` in the terminal.
    - Alternatively, download from the official website.
  - **Linux:**
    - Use the package manager specific to your distribution (e.g., `sudo apt-get install git` for Ubuntu).

Setting Up Git:

- ### Configuring Username and Email:

Run the following commands to set your identity globally:
bash

```
git config --global user.name "Your Name"

git config --global user.email "youremail@example.com"
```

  - *These settings associate your name and email with your commits, making it easier to track who made which changes.*
- *(git config --global --list : this will check if the configurations has been made)*

Basic Git Commands

Creating a Git Repository:

- ➔ **git init**

  Initializes a new Git repository in the current directory.

  Converts an existing project into a Git repository by creating a .git directory.

- ➔ **git clone**

  Clones an existing repository from a remote server to your local machine.

  Syntax: git clone <https://github.com/OgbuEsther/Easy_HR_BE.git>

Basic Git Commands

Staging and Committing:

➔ git add

Adds changes from the working directory to the staging area.

Files in the staging area are ready to be committed.

Syntax: git add <file_name> or git add . to stage all changes.

➔ git commit

Records changes in the staging area to the repository.

Commits should include a descriptive message using the -m option.

Syntax: git commit -m "Commit message"

Basic Git Commands

Staging and Committing:

- ➔ **git status**

  Shows the current state of the working directory and staging area.

  Indicates which changes have been staged, which haven't, and which files aren't being tracked by Git.

- ➔ **git log**

  Displays the commit history, including commit IDs, author information, and commit messages.

  Useful for reviewing changes and tracking project progress

Basic Git Commands

Working with Remote Repositories:

- ➔ **git remote add**

  Links a local repository to a remote repository, allowing you to push and pull changes.

  Syntax: git remote add origin <repository_url>

- ➔ **git push**

  Sends committed changes from the local repository to a remote repository.

  Syntax: git push origin master (assuming the default branch is named master or main).

Basic Git Commands

Working with Remote Repositories:

➔ git pull

Fetches changes from the remote repository and merges them into the local repository.

Useful for keeping your local repository up to date with remote changes.

Syntax: git pull origin master

Understanding Git Repository Structure

Working Directory, Staging Area, and Git Directory:

- ➔ **Working Directory**

  The directory where you modify files. Changes here can be tracked by Git.

- ➔ **Staging Area**

  Also known as the index, it holds changes that are ready to be committed. Files in this area have been added using git add.

- ➔ **Git Directory**

  The repository itself, which contains all of Git's internal data structures, including the history of all changes.

.

Understanding Git Repository Structure

The Lifecycle of a File in Git:

➔ Untracked

The file is new and not yet tracked by Git.

➔ Tracked

The file is being tracked by Git. It can be in one of three states:

- ➔ **Modified:** Changes have been made but not yet staged.
- ➔ **Staged:** Changes have been added to the staging area.
- ➔ **Committed:** Changes have been committed to the repository.

.

Q&A SESSIONS

Q&A SESSIONS

I WILL ATTEND TO YOUR QUESTIONS NOW, I HOPE YOU ENJOYED THE CLASS

QUESTIONS

Which command is used to initialize a new Git repository?

- a) `git start`
- b) `git create`
- c) `git init`
- d) `git setup`
- **Answer:** c

What does the `git push` command do?

- a) Pulls changes from a remote repository.
- b) Pushes changes to a local repository.
- c) Pushes changes to a remote repository.
- d) Merges changes from different branches.
- **Answer:** c

QUESTIONS

Which command shows the commit history in Git?

- a) `git log`
- b) `git history`
- c) `git commits`
- d) `git view`
- **Answer:** a

What is the purpose of the `git add` command?

- a) To add a new file to the repository.
- b) To add changes to the staging area.
- c) To commit changes directly to the repository.
- d) To add a remote repository.
- **Answer:** b

QUESTIONS

The `git clone` command is used to create a new repository from an existing one.

- **Answer:** True

The staging area is where changes are stored before being committed to the repository.

- **Answer:** True

The `git pull` command is used to push changes to a remote repository.

- **Answer:** False

You can configure your Git username and email using the `git setup` command.

- **Answer:** False

QUESTIONS

Explain the difference between `git commit` and `git push`.

- **Answer:** `git commit` saves your changes to the local repository, while `git push` sends those committed changes to a remote repository.

Why is it important to configure your Git username and email?

- **Answer:** Configuring your Git username and email is important because this information is associated with every commit you make, helping to identify the author of the changes.

What happens when you run `git status`?

- **Answer:** `git status` displays the state of the working directory and the staging area, showing any changes that have been staged, modified, or untracked.

Hands-On Practice:

- Students will initialize a Git repository, create and modify files, stage changes, commit them, and push the changes to a remote repository.
- Use the commands taught to explore the Git repository structure and understand the flow of changes.